Health Checks and Self-Healing

let the platform fix the 3 a.m. failures

Watch the 5-min video

In one line: liveness probes restart stuck containers, readiness probes take instances out of traffic without restarting them, startup probes protect slow starters; controllers then keep pushing the actual state toward the desired state, so routine failures heal without waking anyone.

1 Running, but not working

At 3 a.m. a service deadlocks. The process is still running and the port is open, so a simple "is it up?" check says yes, while every request hangs. Nobody is awake to restart it. A good platform notices and fixes this itself in under a minute: that's **self-healing**, and it starts with the right **health checks** (the endpoints themselves are covered in lesson 68).

2 Three probes, three questions

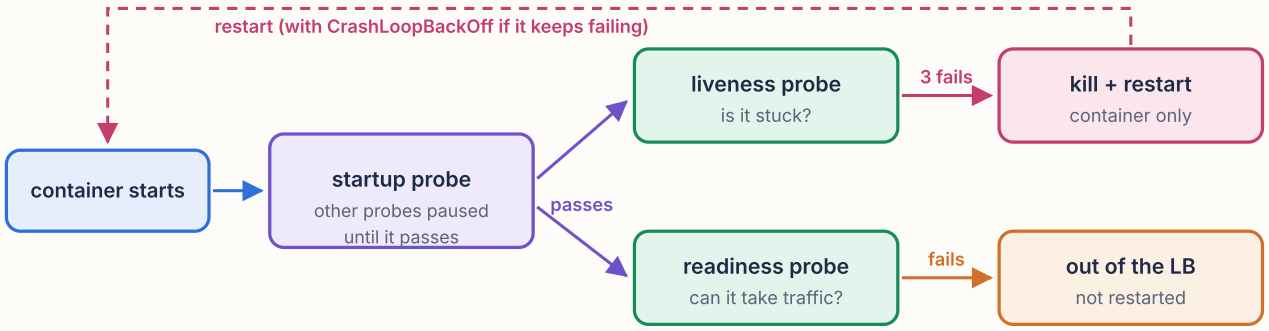


Figure 1. Kubernetes probes. Liveness failure restarts the container; readiness failure only removes the pod from Service endpoints.

Probe	Question	On failure	Should check
Liveness	is this process stuck?	container killed and restarted	only local: can the process answer at all
Readiness	can it serve traffic now?	removed from load balancing	what *this* instance needs: warm cache, loaded model
Startup	has it finished booting?	keeps waiting, then restarts	same endpoint as liveness, with a long allowance

Never check the database in a liveness probe. If the database goes down, every pod fails liveness and the platform restarts the whole fleet, which fixes nothing and adds a cold-start storm on top. Check shared dependencies in readiness carefully (if all pods go unready, nothing serves), or not at all.

3 Self-healing beyond restarts

- A crashed container is restarted by the kubelet; a dead node's pods are **rescheduled** on healthy nodes.
- Cloud auto scaling groups **replace** instances that fail health checks.
- Controllers run a **reconciliation loop**: desired state (5 replicas) vs actual (4), then fix the difference, forever.
- Restarts fix transient problems only. A pod that crashes on startup restarts with exponential backoff, 10 s, 20 s, 40 s... capped at 5 minutes, shown as `CrashLoopBackOff`; that needs an alert and a human.

4 Tuning the probes

`periodSeconds` sets how often the probe runs and `failureThreshold` how many failures in a row trigger action; Kubernetes defaults are 10 s and 3, with a 1 s `timeoutSeconds`. Too aggressive and a 2-second GC pause kills a healthy pod; too lax and a dead pod serves errors for minutes. Time to act ≈ period × threshold, so 10 s × 3 ≈ 30 s.

WORKED EXAMPLE: A DEADLOCKED JAVA SERVICE

Two threads wait on each other forever. The port is open, but `/healthz` runs on the same thread pool and stops answering. Liveness is `httpGet /healthz :8080, periodSeconds: 10, failureThreshold: 3`. After three failures, about 30 s, Kubernetes kills the container and starts a fresh one. Readiness failed too, so the pod was already out of the load balancer. Users saw a few slow requests, not an outage, and the rising restart count tells the team to find the bug.

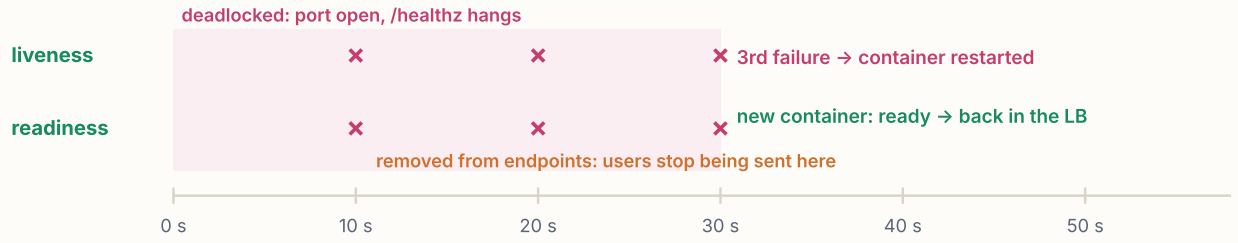


Figure 2. The deadlock heals itself in about 30 s, and readiness keeps users away from the stuck pod meanwhile.

COMMON MISTAKES

- Checking the database or other shared dependencies in liveness: one outage restarts the whole fleet.
- A liveness endpoint served by a separate thread, so it says "alive" while the worker pool is deadlocked.
- No startup probe for a slow-starting app, so liveness kills it before it finishes booting.
- Treating restarts as the fix: frequent restarts hide a bug that still needs an alert.

5 Key terms

Liveness probe: fails → container restarted

Readiness probe: fails → pod removed from traffic, not restarted

Startup probe: holds other probes until the app has booted

Reconciliation loop: controller drives actual state toward desired state

CrashLoopBackOff: restart delay doubling to a 5-minute cap

failureThreshold: consecutive failures before acting

6 Interview questions

Q1. What is the difference between liveness and readiness? BASIC

Liveness asks whether the process is stuck, and failure restarts the container; readiness asks whether it can serve now, and failure only removes it from the load balancer.

Q2. What is a startup probe for? BASIC

Slow-starting apps: liveness and readiness are paused until it passes, so the app isn't killed while still booting.

Q3. How do orchestrators self-heal applications? SCENARIO

They run liveness, readiness and startup probes, restart containers that fail liveness, pull unready pods from traffic, reschedule pods from dead nodes, and keep reconciling actual state toward desired state.

Q4. A pod is in CrashLoopBackOff. What does it mean and what do you do? SCENARIO

It crashes on start and Kubernetes is backing off restarts up to 5 minutes; read the logs of the previous container, fix the config or bug, and roll back if it came from a deploy.

Q5. Why shouldn't a liveness probe check the database? TRICKY

A database outage would fail liveness on every pod and trigger a fleet-wide restart that fixes nothing. Check dependencies in readiness, or not at all.

Q6. Can readiness probes cause an outage? TRICKY

Yes: if every pod checks a shared dependency and it fails, all pods go unready and nothing serves, even requests that didn't need it.

Next: SD-K-010 · Redundancy and Replication backendroadmap.com